

Netflix Domain Hackathon: Object-Oriented Design Mastery (Java)

A structured, enterprise-grade challenge designed to transform students into software architects — through progressive OOP concepts applied to a real-world streaming platform simulation.

DEVELOPED BY TALENCIAGLOBAL

JAVA · OOP · ARCHITECTURE

Why This Hackathon Matters

The global streaming industry is projected to surpass **\$330 billion by 2030**, with platforms like Netflix employing thousands of software engineers who design systems handling over **250 million subscribers** and **3+ billion hours** of content streamed weekly. The domain models powering these platforms are built on the very OOP principles this hackathon teaches.

This challenge is intentionally designed to force students to think like software architects rather than just coders — progressing from basic class design to enterprise-level interface-driven composition.

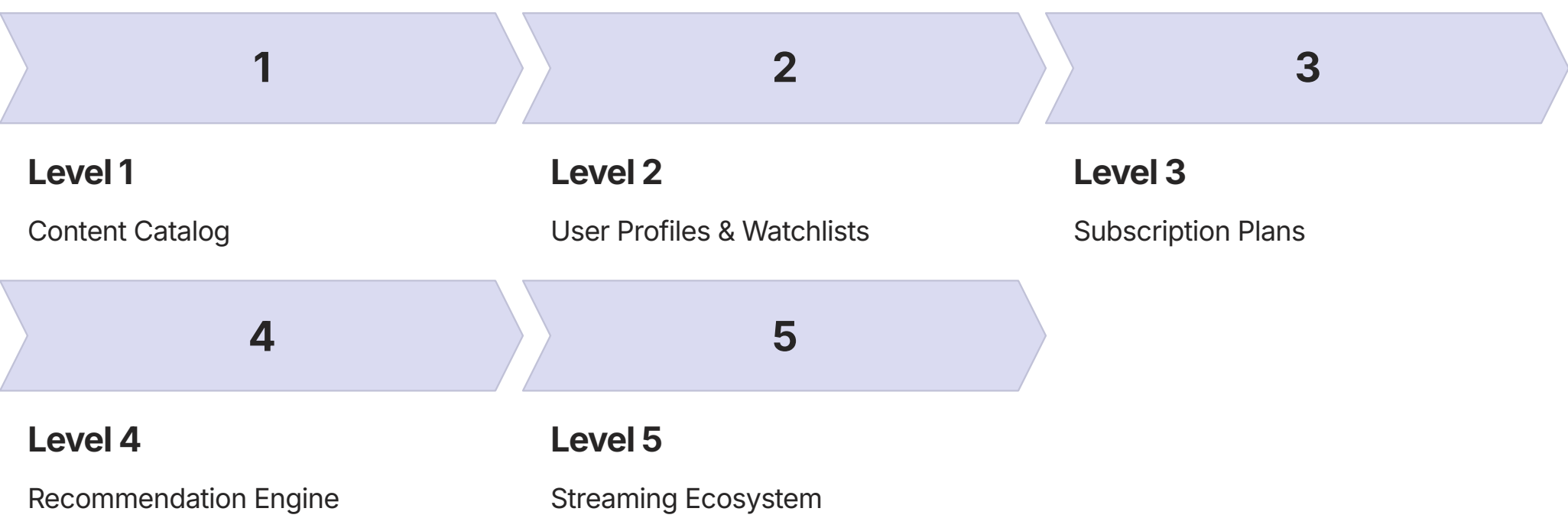
-  Over **70% of enterprise Java codebases** rely on OOP design patterns including delegation, composition, and polymorphism — the exact skills tested here.

Concepts Covered

- Classes & Objects
- Encapsulation & Access Modifiers
- Constructors & Constructor Injection
- HAS-A Relationships
- Collections of Objects
- Delegation & Composition
- Inheritance & Polymorphism
- Abstract Classes & Interfaces

Hackathon Structure at a Glance

Five progressively challenging levels mirror the real-world evolution of a streaming platform's architecture — from a simple content catalog to a fully delegated, interface-driven streaming ecosystem.



5

Challenge Levels

From Intermediate to Expert

13

OOP Concepts

Tested across all levels

10

Evaluation Areas

Weighted scoring criteria

100%

Industry Relevance

Mirrors real platform design

Level 1 — Netflix Content Catalog

INTERMEDIATE

Netflix hosts over **17,000 titles** across its global catalog. This level simulates the foundational content management system — the backbone of any streaming platform. Students build the data model that makes content discoverable, manageable, and encapsulated.



Level 1 — Class Design Requirements

Class: Content

The atomic unit of the Netflix catalog. All fields must be strictly private, enforcing encapsulation from the ground up.

contentId

Unique identifier — private, read-only via getter

title

Content title — private, getter only

genre

Genre classification — private, getter only

durationMinutes

Runtime in minutes — private, getter only

Class: Catalog

Manages a `List<Content>` — a HAS-A relationship demonstrating object aggregation at scale.

Add Content

Append new titles to the catalog

Remove Content

Delete titles by reference or ID

Search by Title

Retrieve content by name lookup

Display All

Enumerate the full catalog

- ❏ **Key Constraint:** Constructor initialization is mandatory. No direct field access permitted. Getter methods are the only sanctioned access path — mirroring production-grade Java bean design.

Level 1 — Concepts Tested



Classes & Objects

Defining blueprints and instantiating real-world entities as Java objects with identity and state.



Constructors

Mandatory initialization patterns ensuring objects are always in a valid, consistent state upon creation.



Encapsulation

Hiding internal state behind private fields and controlled getter interfaces — the first pillar of OOP.



Collection HAS-A

Modeling the Catalog's ownership of a `List<Content>` — a foundational aggregation relationship.

Level 2 — User Profiles & Watchlists

INTERMEDIATE+

Netflix supports up to **5 profiles per account**, each with independent watchlists and viewing histories. This level models that multi-profile architecture, introducing constructor injection and object aggregation as first-class design tools.

Level 2 — Class Design & Constraints

Class: User

- **userId** — unique account identifier
- **name** — account holder name
- HAS-A `List<Profile>`
- Can create and delete profiles

Class: Profile

- **profileName** — display name
- **ageCategory** — content rating filter
- HAS-A `List<Content>` watchList
- Add, remove, and display watchlist items

Constructor Injection Requirement

Profiles must be passed through constructors. No setters are permitted for profile assignment.

```
User user = new User(  
    "U101",  
    "John",  
    profiles  
);
```

This enforces immutable object graphs and mirrors dependency injection patterns used in Spring Framework — the most widely adopted Java enterprise framework globally.

1

Constructor Injection

Objects receive dependencies at creation time

2

Object Aggregation

User owns Profiles; Profile owns Content

3

Composition Thinking

Modeling real-world ownership hierarchies

4

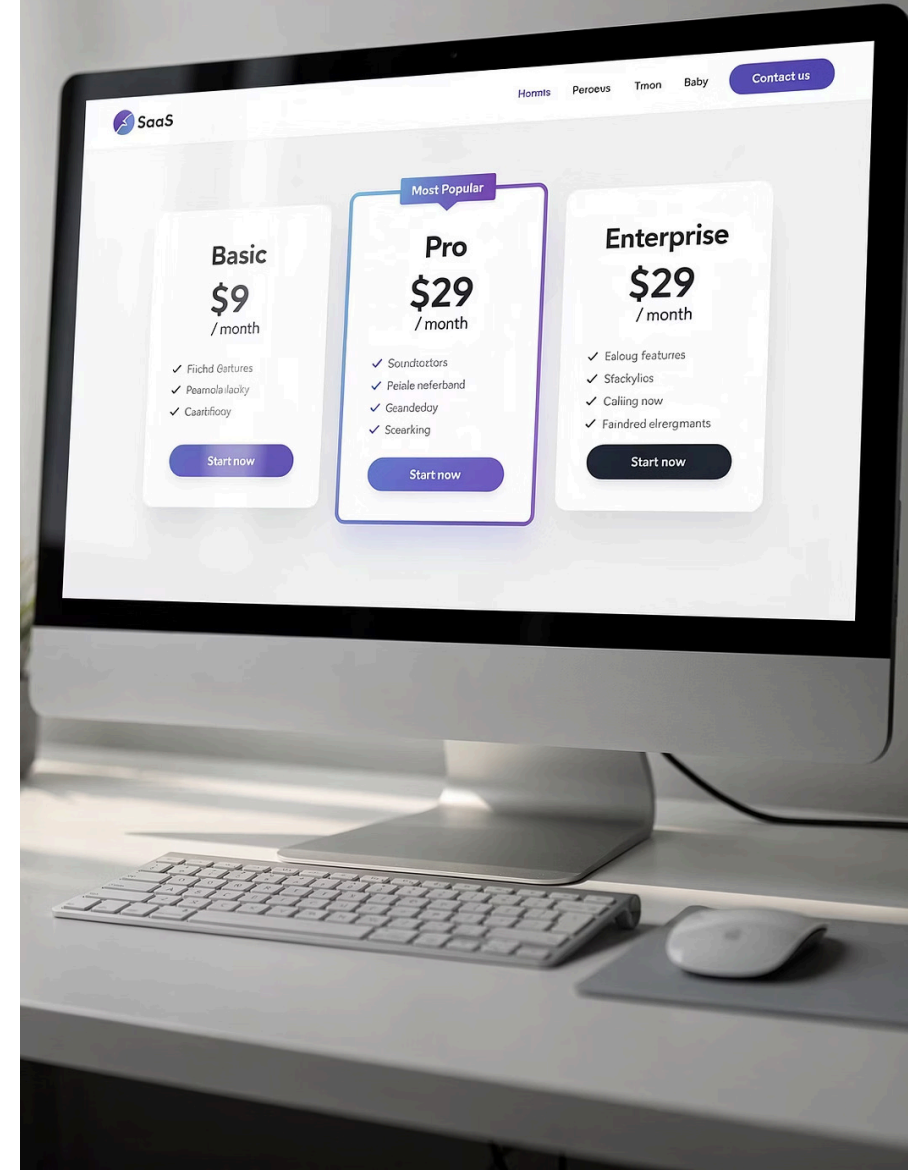
Real-World Modeling

Mirrors Netflix's actual account architecture

Level 3 — Premium, Mobile & Family Plans

ADVANCED

Netflix's subscription revenue exceeded **\$33 billion in 2023**, driven by a tiered plan structure. This level models that architecture using abstract classes, inheritance, and runtime polymorphism — enabling the system to support new plan types without modifying existing business logic.



Level 3 — Abstract Class & Derived Plans

Abstract Class: SubscriptionPlan

- **planName** — plan identifier
- **monthlyCost** — billing amount
- `displayFeatures()` — abstract method
- `calculateMonthlyCost()` — concrete method

✓ User HAS-A SubscriptionPlan — enabling runtime plan switching without altering business code.

PremiumPlan

4K Ultra HD, 4 screens, all devices. Implements `displayFeatures()` with premium-tier output.

MobilePlan

SD quality, 1 screen, mobile only. Lightweight implementation for cost-sensitive markets.

FamilyPlan

HD quality, 2 screens, shared household. Implements family-specific feature display logic.

```
SubscriptionPlan plan = new PremiumPlan();  
// ... later, without changing business code:  
plan = new FamilyPlan();
```

This pattern — assigning derived types to abstract references — is the essence of **runtime polymorphism** and is foundational to the Open/Closed Principle in enterprise Java design.

Level 4 — Recommendation Engine

ADVANCED+

Netflix's recommendation algorithm influences over **80% of content watched** on the platform, saving an estimated **\$1 billion annually** in customer retention. This level models the delegation architecture behind such a system — ensuring recommendation logic is decoupled from the Profile class entirely.



Level 4 — Interface, Delegation & Strategy Pattern

Interface: RecommendationStrategy

```
List<Content> recommend(  
    Profile profile,  
    Catalog catalog  
);
```

Implementations

GenreBasedRecommendation

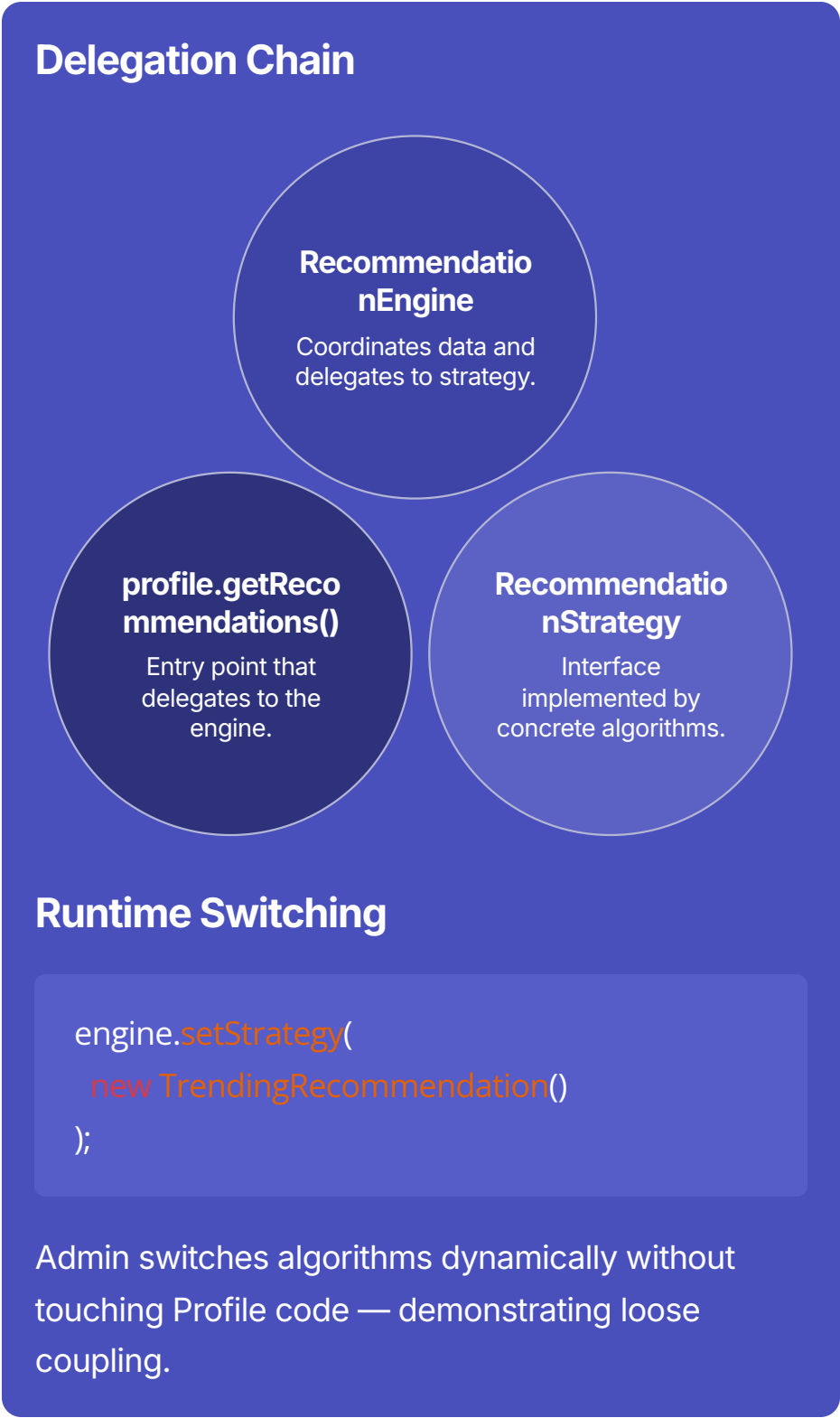
Filters catalog by user's preferred genres

TrendingRecommendation

Returns globally trending content at runtime

WatchHistoryRecommendation

Analyzes past viewing patterns for suggestions



Design Principle: Profile must NOT perform recommendation logic. This separation of concerns mirrors the Strategy Pattern — one of the Gang of Four's 23 foundational design patterns used in virtually every enterprise Java system.

Level 4 — Concepts Tested



Interfaces

Defining contracts that multiple strategies implement independently



Delegation

Profile delegates to Engine; Engine delegates to Strategy



Composition

RecommendationEngine HAS-A
RecommendationStrategy



Strategy Pattern

Algorithms encapsulated and interchangeable at runtime



Loose Coupling

Profile is unaware of which strategy is active



Object Collaboration

Multiple objects cooperate without tight dependencies

Level 5 — Netflix Streaming Ecosystem

EXPERT

Netflix streams to over **2,000 device types** globally — from smart TVs to gaming consoles. This expert-level challenge models that multi-device architecture using interface-based design, composition over inheritance, and a fully delegated playback system that supports future device types without modifying existing code.



Level 5 — Interface & Class Architecture

Interface: PlayableDevice

```
void play(Content content);  
void pause();  
void stop();
```

Implementations

SmartTV

4K playback, large
screen rendering

MobileDevice

Adaptive streaming,
touch controls

LaptopDevice

Browser-based or
native app playback

GamingConsole

Controller-driven, HDR-
enabled playback

Class: StreamingSession

The central orchestrator of a viewing session. Holds three HAS-A relationships:

- HAS-A **PlayableDevice**
- HAS-A **Profile**
- HAS-A **Content**

```
StreamingSession(  
    profile,  
    device,  
    content  
);  
  
session.start();  
// Internally: device.play(content);
```

- ❏ The session delegates playback entirely to the device — it never contains playback logic itself.

Level 5 — DeviceManager & Architectural Justification

Class: DeviceManager

Manages `List<PlayableDevice>` with full lifecycle control:

- Device registration
- Device removal
- Device lookup by type or ID
- Active streaming count query

Architectural Justification Required



Why Interface?

Storing `SmartTV`, `MobileDevice` directly in `StreamingSession` would require code changes for every new device type — violating the Open/Closed Principle.



Why Delegation?

Placing playback code in `StreamingSession` violates Single Responsibility — the session manages state, not device behavior.

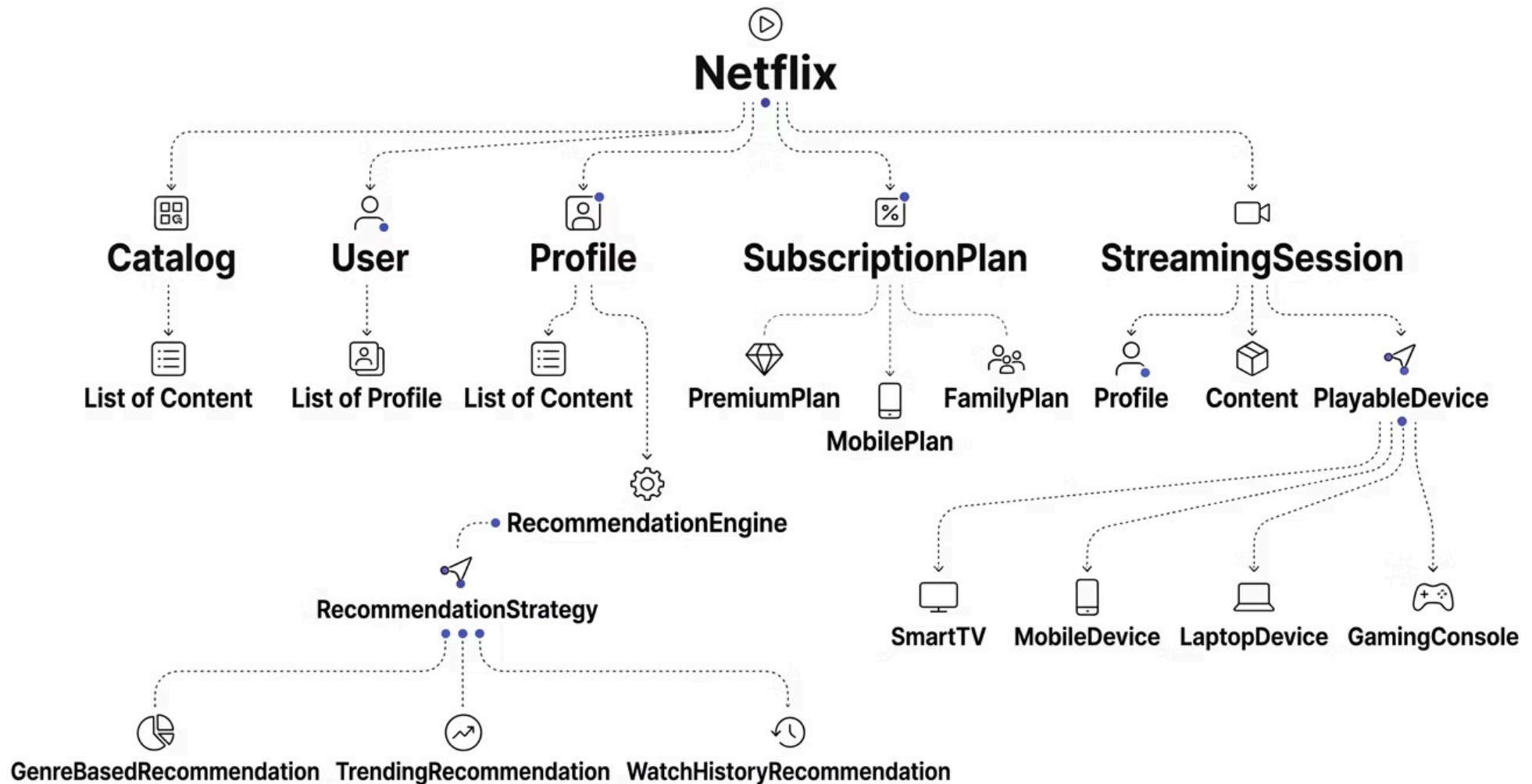


Why Composition?

Inheriting `StreamingSession` from `Device` creates a false IS-A relationship — a session is not a device; it uses one.

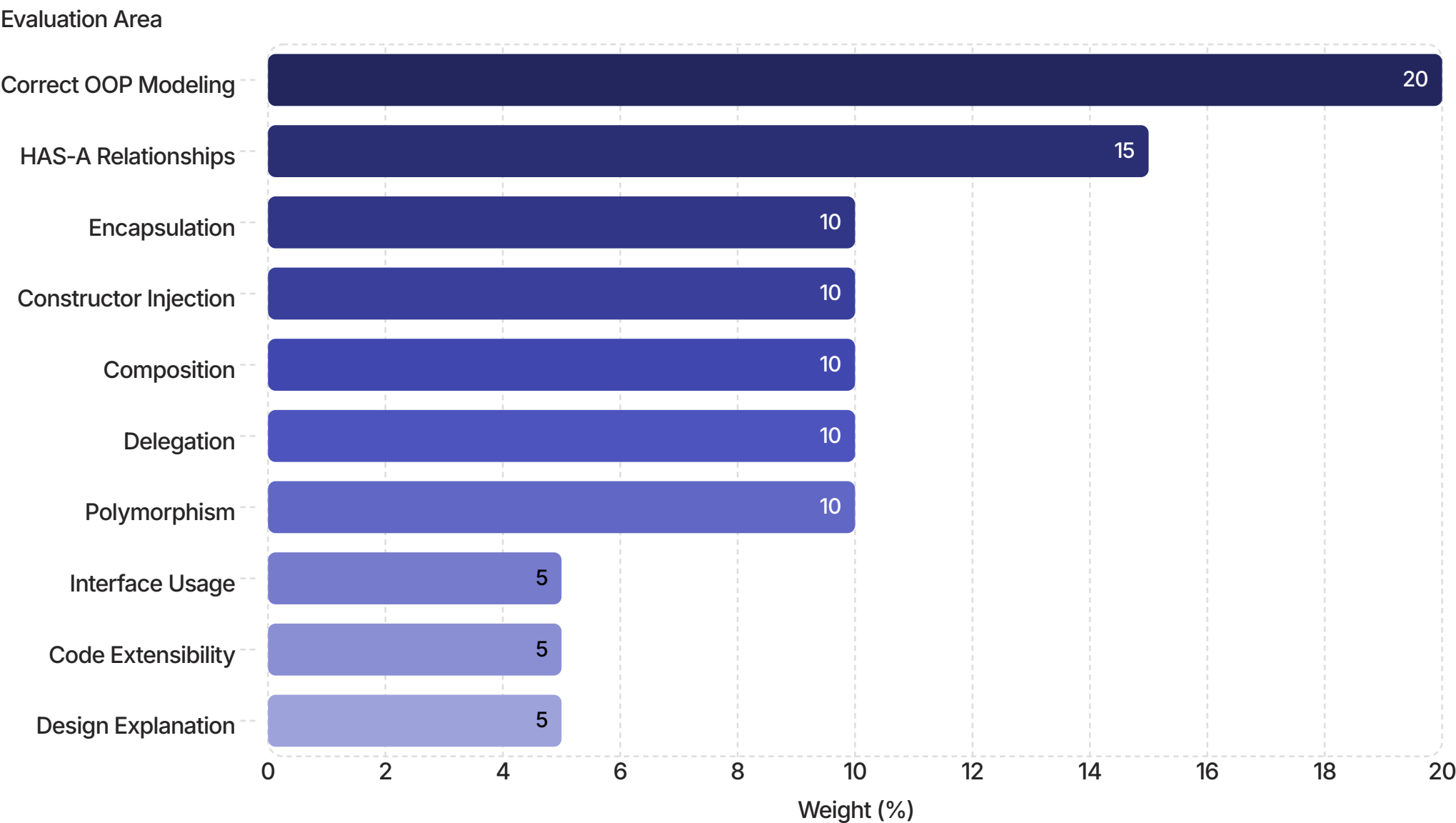
Final Architecture — Complete System Overview

The complete Netflix simulation produces an enterprise-grade object graph where every relationship is intentional, every dependency is injected, and every abstraction is justified by real-world design principles.



Evaluation Criteria & Scoring

Student submissions are assessed across ten weighted dimensions, reflecting the priorities of enterprise Java development teams. Design explanation carries weight equal to interface usage — reinforcing that architectural reasoning is as valuable as implementation.



OOP Modeling carries the highest weight at 20%, reflecting that structural correctness is the foundation upon which all other design qualities depend.



Industry Relevance — Why This Architecture Matters

\$330B

Streaming Market

Projected global value by 2030 — built on OOP architectures like this

250M+

Netflix Subscribers

Served by domain models structurally identical to this simulation

80%

Content Discovery

Of Netflix viewing driven by recommendation engines using delegation patterns

2000+

Device Types

Supported via interface-based design — exactly the pattern taught in Level

5

- ✔ Students who complete all five levels will have implemented — in simplified form — the same architectural patterns used by Netflix, Amazon Prime Video, Disney+, and every major enterprise Java platform in production today.

Begin Your Architect Journey

This hackathon is not merely a coding exercise — it is a structured pathway from object-oriented fundamentals to enterprise-grade design thinking. Each level builds upon the last, culminating in a system that demonstrates mastery of the thirteen core OOP concepts that define professional Java development.

The best software architects do not just write code that works — they design systems that are extensible, maintainable, and explainable. This hackathon trains exactly that mindset.

Start with Level 1 Master encapsulation and the Content Catalog before advancing	Progress Deliberately Each level introduces one new architectural concept — do not skip ahead
Justify Every Decision Design explanation is scored — articulate why, not just how	Think Like an Architect The final architecture should reflect enterprise-grade OOP thinking